Deinterleave Even and Odd Indices in String

Question 2: 2025 Jan OPPE 1 - Set 1

Deinterleave Even and Odd Indices

The Problem

Write a function `deinterleave(s)` that takes a string and returns a new string where all characters at even indices appear before all characters at odd indices.

> **Note:** This is a function-only question. No input statements or print statements required—just complete the function.

Understand the Question Clearly

Key Distinction

This problem is **NOT** about even numbers—it's about **even indices**.

Remember the crucial difference:

> Index ≠ Value

Example Breakdown

String: "abcdef"
Indices: 012345

- Even indices ↣ positions 0, 2, 4 ↣ characters `a`, `c`, `e`
- Odd indices ↣ positions 1, 3, 5 ↣ characters `b`, `d`, `f`

What Does Deinterleave Mean?

The process involves three distinct steps to reorganize the string by position:

Collect Even-Index Characters

Gather all characters at positions 0, 2, 4, ...

Collect Odd-Index Characters

Gather all characters at positions 1, 3, 5, ...

Combine the Parts

Even part + Odd part = Final result

Step-by-Step Example

```
"abcdef"
Even indices → a c e
Odd indices  → b d f
Final result → "acebdf"
```

More Examples

Understanding the pattern through various inputs:

Input	Even Index Chars	Odd Index Chars	Output
"abcdef"	"a", "c", "e"	"b", "d", "f"	"acebdf"
"12345"	"1", "3", "5"	"2", "4"	"13524"
"python"	"p", "t", "o"	"y", "h", "n"	"ptonyhn"

Important Insight

We are **NOT** sorting the characters. We are **NOT** modifying individual characters. We are simply **reorganizing by position**.

How to Access Even Indices in Python

Python Slicing to the Rescue

Python's powerful slicing syntax makes collecting even indices incredibly simple. The key insight:

> `s[::2]`

Breaking Down the Syntax

- **Start:** Index 0 (default start)
- **Stop:** End of string (default stop)
- **Step:** 2 (jump by 2 positions)

Visual Example

```
"abcdef"[::2]
Indices: 012345
         ↑ ↑ ↑
         a c e
Result: "ace"
```

The slice systematically visits every even index: 0, 2, 4, ...

How to Access Odd Indices

The Pattern

Similar to even indices, but we start at a different position.

The Syntax

`s[1::2]`

The Result

Starts at index 1, steps by 2

Code Example

```
s = "abcdef"
odd_chars = s[1::2]
print(odd_chars)
# Output: "bdf"
```

How It Works

- **Start:** Index 1 (first odd index)
- **Step:** 2 (skip one, take next)
- **Result:** Positions 1, 3, 5, ...

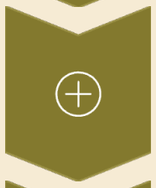
The Elegant Solution

Combine both slicing techniques into a single, concise function:

```
def deinterleave(s: str) -> str:
    return s[::2] + s[1::2]
```

Even Part

`s[::2]`Collects indices 0, 2, 4...



Plus

Concatenation operator

Odd Part

`s[1::2]`Collects indices 1, 3, 5...

> ### That's it!
>
> No loops required. No manual indexing. No temporary variables. Just pure Python elegance.

Naive vs Pythonic

What Many Students Do

- Write an explicit loop
- Check `index % 2 == 0`
- Manually append to two strings
- Combine results at the end

Example of Overcomplication

```
even = ""
odd = ""
for i in range(len(s)):
    if i % 2 == 0:
        even += s[i]
    else:
        odd += s[i]
return even + odd
```

It works—but it's unnecessarily complex and verbose.

What You Should Do

- Use Python slicing syntax
- Leverage built-in functionality
- Write concise, readable code

The Pythonic Approach

```
def deinterleave(s):
    return s[::2] + s[1::2]
```

✓ Cleaner
✓ Shorter
✓ Harder to mess up in exams
✓ **More Pythonic**

Why Slicing Is Better

Less Code

1 line instead of 8 lines. Reduces cognitive load and file size.

Fewer Bugs

No manual index management. Eliminates off-by-one errors and logic mistakes.

More Pythonic

Signals you truly understand Python idioms and built-in capabilities.

Additional Benefits

Slicing is also **faster** because it's implemented in C, and it's **clearer** to readers who understand Python conventions. It transforms a multi-step algorithm into a single, expressive operation.

Engineering Best Practices

Apply these principles to future problems:

01

Recognize Common Patterns

Even indices ↠ step = 2

Odd indices ↠ start = 1, step = 2

02

Avoid Manual Loops When Built-ins Exist

Slicing, list comprehensions, and built-in functions often outperform explicit loops.

03

Keep Logic Flat and Clear

Minimize nesting and temporary variables.

Final Logic Applied

```
return s[::2] + s[1::2]
```

- **Pattern Recognition:** Spot slicing opportunities
- **Leverage Built-ins:** Use Python's powerful tools
- **Clean Output:** Return, don't print